



PLAGUE CONTAINMENT

A Grid-Based Epidemic Strategy Game

Nelson David Molina · Brayan Estiven Aguirre · Juan Sneyder Méndez

Computer Sciences I · Team 16 · Semester 2026-I

Universidad Distrital Francisco José de Caldas

THE PROBLEM

How do you stop a plague with limited resources?

Epidemic containment is a dual challenge:

LOCAL decisions — which district to protect right now?

GLOBAL planning — how to surround the infection without cutting off healthy areas?

Our game puts the player in charge of both.



Vaccination

Protect the most vulnerable district before the plague reaches it



Quarantine

Surround the infected zone to stop the spread completely

THE GAME

An 8×8 city under siege

3	5	2	7	4	6	1	3
4	6	8	5	3	7	2	5
2	4	6	8	9	7	3	4
5	3	4	7	6	5	8	2
7	1	9	2	5	3	6	4
3	8	4	6	1	7	2	9
6	2	5	3	8	4	1	7
1	9	3	5	2	8	4	6

Healthy

Infected

Vaccinated

Quarantined

Greedy Tip

Game Rules

- The city**

64 districts on an 8×8 grid, each with a risk level (1-10)
- Each turn**

The plague spreads to healthy neighbors of infected districts
- Player actions**

Vaccinate one district (blocks infection permanently)

Apply quarantine plan (isolates the infected zone)

Skip turn (do nothing)
- Win condition**

Fully contain the plague before all districts are infected
- Lose condition**

The plague consumes the entire city

OUR SOLUTION

Two algorithms, two strategies



Greedy Vaccination

"Always vaccinate the most dangerous district first"

1. Find all healthy districts next to infected ones
2. Pick the one with the highest risk level
3. Recommend it to the player

Why it works: The most urgent threat IS the highest-risk neighbor. Local optimality matches the immediate goal.



Backtracking Quarantine

"Find the perfect containment ring"

1. Identify border districts around infected zone
2. Try quarantining each combination
3. Check: enclosed? + all healthy connected?
4. If not → undo (backtrack) and try another

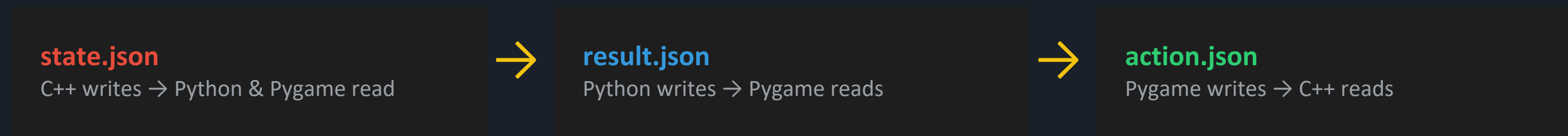
Why backtracking: A greedy choice could isolate a healthy zone. We need to explore all valid options.

THE ARCHITECTURE

Three layers, one language-neutral bridge



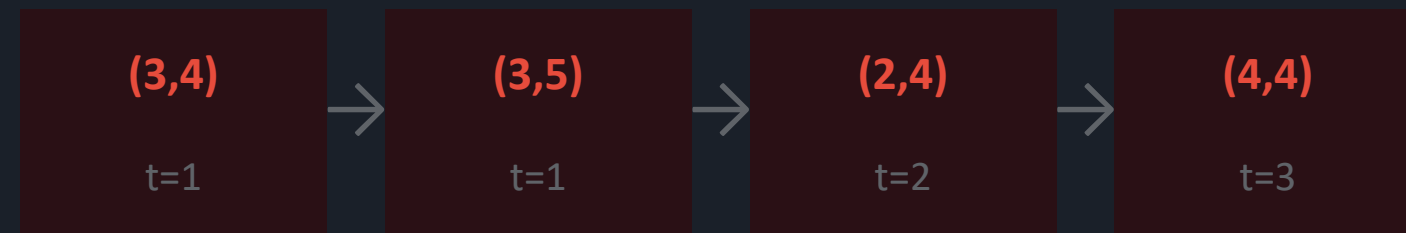
JSON Bridge — How they communicate



The engine under the hood

Linked List

Infection Chain



Each node stores:

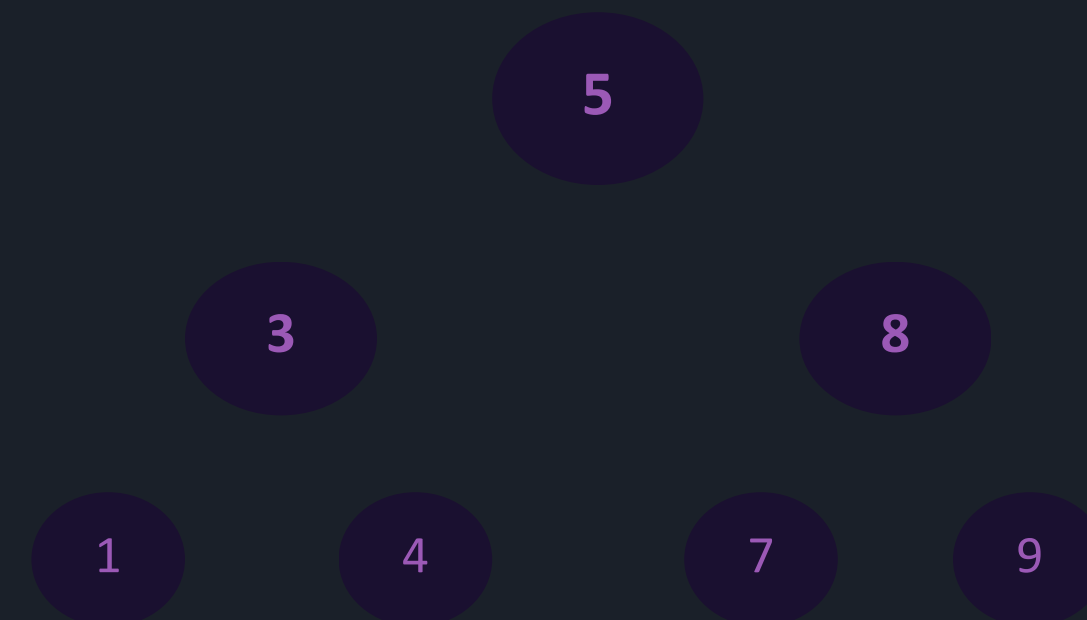
Position (row, col) + turn infected

Why linked list?

Infection grows dynamically — we don't know how many districts will be infected. $O(1)$ insertions at tail.

AVL Tree

District Risk Index



Sorted by risk level

Find highest-risk district in $O(\log n)$

Why AVL over BST?

Self-balancing guarantees $O(\log n)$ even with sorted insertions. A normal BST could degrade to $O(n)$.

TEAM & NEXT STEPS

From design to a playable game



Engine Developer

**Nelson David
Molina Ramos**

C++ data structures:
Linked List + AVL Tree



Algorithm Developer

**Brayan Estiven
Aguirre Aristizabal**

Python algorithms:
Greedy + Backtracking



UI & Integration

**Juan Sneyder
Méndez Gil**

Pygame interface,
JSON bridge, architecture

What's Next

Week 9-12: Implement C++ engine (linked list, AVL tree, plague spread)

Week 10-14: Build Python algorithms (greedy + backtracking) and test with mock JSON

Week 12-15: Develop Pygame interface, integrate all components via JSON bridge

Week 16: Final demo, polished game, technical report, GitHub repo